

Kapitel 6

Steuerung des Programmablaufs

6.1 Anweisungen und Blöcke

Mit **while**, **if** und **return** haben wir bereits drei Anweisungen kennengelernt, mit denen sich der Ablauf eines Programms beeinflussen lässt. In diesem Abschnitt wollen wir uns die Anweisungen zur Steuerung des Programmablaufs im Zusammenhang ansehen.

Die einfachsten C-Anweisungen sind die *Ausdrucksanweisungen*. Sie bestehen aus einem Ausdruck, dem ein Semikolon folgt.

Beispiele:

```
a = b + c;
++i;
```

Wie bereits erwähnt: Das Semikolon ist in C ein *Abschlusssymbol* und kein *Trennsymbol*, wie etwa in Pascal.

Mehrere Anweisungen können zu einem *Block* zusammengefasst werden, indem man sie in ein Paar *geschweifte Klammern* (**{ }**) einschließt. Ein solcher Block ist syntaktisch einer einzelnen Anweisung äquivalent. Wir haben Blöcke schon bei **while**- und **if**-Anweisung kennengelernt. Achtung: Nach der geschweiften Klammer, die einen Block abschließt, steht *kein* Semikolon!

Ein Block darf auch leer sein, d.h. zwischen seinen Klammern brauchen keine Anweisungen zu stehen. Er ergibt dann eine *leere Anweisung*, d.h. eine Anweisung, die nichts bewirkt. Eine leere Anweisung kann man auch durch aufeinanderfolgende Semikolons oder ein Semikolon als ersten Eintrag in einem Block erzeugen. Auch wenn die leere Anweisung auf den ersten Blick ziemlich nutzlos erscheint, ist sie gelegentlich doch sehr hilfreich: Wir haben bereits ein Beispiel (leeren der Eingabe) gesehen, bei dem die gesamte Operation einer Schleife in der Bedingung steckte, so dass der Anweisungsteil der Schleife nur noch aus einer leeren Anweisung bestand:

```
while (getchar () != '\n') {
}
```

Vor der ersten Anweisung in einem Block können Variablenvereinbarungen und beliebige andere Deklarationen stehen. Für die Funktion **main** haben wir das bereits kennengelernt. Gleiches gilt aber auch z.B. für den Block einer Schleife.

6.2 Die if-Anweisung

Die **if**-Anweisung wurde bereits informell eingeführt. Ihre vollständige Syntax ist

```
if (bedingung)
    anweisung1
[ else
    anweisung2 ]
```

Was sie bewirkt, sollte inzwischen geläufig sein. Interessanter sind die zusätzlichen formalen Möglichkeiten:

Formal wird hinter dem `if` selbst und ggf. auch hinter dem `else` *genau eine* Anweisung verlangt. Diese Anweisung kann ein Block sein, wie in den bisherigen Beispielen, braucht es aber nicht. Speziell die `if`-Anweisung, mit der der bedingte Ausdruck beschrieben wurde, könnte man also sehr viel kürzer schreiben:

```
if (bedingung)
    x = ausdruck1;
else
    x = ausdruck2;
```

Zum anderen kann man das Schlüsselwort `else` samt nachfolgender Anweisung weglassen, wenn diese Anweisung leer ist – in der Syntaxbeschreibung durch die eckigen Klammern angedeutet. Das ist oft bequem, führt gelegentlich bei geschachtelten `if`-Anweisungen aber auch zu Problemen. Wir betrachten dafür ein Beispiel:

```
if (n > 0)
    if (n % 2)
        printf("positiv und ungerade\n");
    else
        printf("nicht positiv\n");
```

So, wie die Anweisung aufgeschrieben ist, suggeriert sie folgenden Ablauf

- Wenn `n` positiv und ungerade ist, wird die entsprechende Meldung geschrieben.
- Wenn `n` positiv und gerade ist, passiert nichts.
- Wenn `n` nicht positiv ist, wird die entsprechende Meldung geschrieben.

Tatsächlich ist der Ablauf anders:

- Wenn `n` positiv und ungerade ist, wird, wie oben, die entsprechende Meldung geschrieben.
- Wenn `n` positiv und gerade ist, wird die Meldung “nicht positiv” geschrieben.
- Wenn `n` nicht positiv ist, passiert nichts.

Woran liegt das? Den Compiler interessiert es überhaupt nicht, wie man sein Programm aufschreibt; das Einrücken dient nur dazu, dem Leser den Überblick über die Struktur eines Programms zu erleichtern. Die Zuordnung eines `else` zu einem `if` durch den Compiler erfolgt vielmehr so, wie die Zuordnung einer schließenden zu einer öffnenden Klammer: Wird eine schließende Klammer gefunden, wird rückwärts gesucht, bis die letzte, passende öffnende Klammer gefunden wird, der noch keine schließende Klammer zugeordnet ist.

Wenn man mit einem Editor arbeitet, der den Quelltext automatisch einrückt, kann es einem übrigens nicht so leicht passieren, dass vermeintliche und tatsächliche Zuordnung nicht übereinstimmen.

Im Beispiel gibt es drei „Reparaturmöglichkeiten“. Die erste ist formaler Natur: Man vervollständigt die geschachtelte `if`-Anweisung durch einen `else`-Zweig:

```
if (n > 0)
    if (n % 2)
        printf("positiv und ungerade\n");
    else
        ;
else
    printf("nicht positiv\n");
```

Die zweite ist ebenfalls formaler Natur: Man macht die geschachtelte `if`-Anweisung zu einem Block – es ist klar, dass mit dem Ende eines Blocks eine `if`-Anweisung abgeschlossen sein muss, auch wenn noch kein `else` für sie da war:

```
if (n > 0) {
    if (n % 2)
        printf("positiv und ungerade\n");
}
else
    printf("nicht positiv\n");
```

Schließlich kann man, zumindest in diesem Beispiel, auch die Logik „umdrehen“:

```
if (n <= 0)
    printf ("nicht positiv\n");
else
    if (n % 2)
        printf("positiv und ungerade\n");
```

Dieses letzte Beispiel legt eine etwas veränderte Schreibweise nahe: Wenn die Anweisung hinter einem `else` ihrerseits eine `if`-Anweisung ist, liegt es nahe, beide Zeilen zu einer zusammenzufassen:

```
if (n <= 0)
    printf("nicht positiv\n");
else if (n % 2)
    printf("positiv und ungerade\n");
```

Die Form, in der man seine Programme aufschreibt, sollte ja, wie eben bereits angesprochen, schon auf den ersten Blick einen möglichst guten Überblick über die Struktur der Programme erlauben. Dazu dient unter anderem das Einrücken. Wenn allerdings zu viele Stufen oder innerhalb der Stufen zu weit eingerückt wird, schadet das wiederum der Übersichtlichkeit. Oft ist man auch zu Kompromissen gezwungen.

Nach so viel Theorie und formalen Beispielen nun noch ein konkretes Beispiel: Der Radius eines Kreises soll abgefragt und, je nach Wunsch, Kreisumfang oder Kreisfläche ausgegeben werden.

```
#include <stdio.h>

#define PI 3.14159

int main(void) {
    int antwort;
    double r;

    printf("Bitte geben Sie den Radius ein: ");
    scanf("%lf", &r);
    while (getchar () != '\n')
        ;
}
```

```

printf("Soll der Umfang (u/U) oder die Flaeche (f/F) "
      "des Kreises berechnet werden? ");
antwort = getchar();
if (antwort == 'u' || antwort == 'U')
    printf("Umfang: %E\n", 2 * PI * r);
else if (antwort == 'f' || antwort == 'F')
    printf("Flaeche: %E\n", PI * r * r);
else
    printf("Falsche Eingabe!\n");

return 0;
}

```

Quelltext 6.1: Berechnung von Kreisumfang oder Fläche

6.3 Die switch-Anweisung

Was im vorherigen Programmbeispiel realisiert ist, kommt oft vor: Abhängig von einem bestimmten Wert ist nicht nur aus zwei, sondern aus mehr Möglichkeiten auszuwählen. C kennt speziell dafür die **switch**-Anweisung

```

switch (ausdruck) {
    case wert1:
        anweisungfolge1
    case wert2:
        anweisungfolge2
    ...
    case wertN:
        anweisungfolgeN
[ default:
    anweisungfolgeD ]
}

```

Der Ablauf ist so: Der Ausdruck *ausdruck* wird ausgewertet. Ist das Resultat einer der einem **case** folgenden Werte *wert1*, ..., *wertN*, so wird die Ausführung mit der Anweisungsfolge fortgesetzt, die diesem Wert folgt. Kommt der Wert des Ausdrucks nicht vor, wird die Anweisungsfolge hinter **default** ausgeführt, wenn eine **default**-Klausel angegeben ist, bzw. sonst direkt hinter das Ende der **switch**-Anweisung verzweigt.

Vor einem Beispiel zunächst noch einige Anmerkungen zu den Formalien:

- Sowohl der Ausdruck *ausdruck* als auch die Werte *wert1*, ..., *wertN* müssen ganzzahlig sein. Zeichenkonstanten sind also erlaubt.
- Die Werte *wert1*, ..., *wertN* müssen sämtlich verschieden sein.
- Die Werte *wert1*, ..., *wertN* müssen Konstanten oder *konstante Ausdrücke* sein. Unter einem konstanten Ausdruck versteht man einen Ausdruck, dessen Operanden sämtlich Konstanten sind, in dem keine Funktionen aufgerufen werden und in dem keine Operatoren mit Nebeneffekten vorkommen. Der Hintergrund: Konstante Ausdrücke werden bereits durch den Compiler ausgewertet, sind bei der Ausführung des Programms also Konstanten.
- Die Reihenfolge, in der *wert1*, ..., *wertN* und ggf. **default** angegeben werden, ist beliebig. Bei der Reihenfolge, die man wählt, sollte man die Lesbarkeit des Programms allerdings besonders im Auge haben.

- Jeder **case**- und der **default**-Klausel darf eine *Anweisungsfolge* folgen, nicht nur eine einzelne Anweisung. Insbesondere entfällt damit die Notwendigkeit, Anweisungsfolgen durch geschweifte Klammern zu Blöcken zusammenzufassen.

Wir sehen uns jetzt ein Beispiel an. In einem Programm, das mit Wochentagen arbeitet, wird man die Wochentage mit Kennziffern identifizieren, etwa Sonnabend mit 0, Sonntag mit 1, usw.. Will man diese Kennziffern in Klarschrift umsetzen, so kann man das mit einer **switch**-Anweisung tun, auch wenn es gerade für dieses Beispiel günstigere Möglichkeiten gibt, wie wir noch sehen werden:

```
enum TAGE {SONNABEND, SONNTAG, ...};
...
switch (wochentag) {
    case SONNABEND:
        printf("Sonnabend\n");
    case SONNTAG:
        printf("Sonntag\n");
    ...
    case FREITAG:
        printf("Freitag\n");
    default:
        printf("Kennzahl unzuverlässig\n");
}
```

Dieses Beispiel funktioniert für unzulässige Kennzahlen famos – und liefert für zulässige Kennzahlen nicht das beabsichtigte Ergebnis. Ist die Kennzahl etwa **DONNERSTAG**, so erhalten wir die drei Ausgabezeilen

```
Donnerstag
Freitag
Kennzahl unzuverlässig
```

Diese Ausgabe zeigt auch bereits, was schief geht: Wenn die Anweisungsfolge für einen Fall ausgeführt ist, wird nicht automatisch hinter das Ende der **switch**-Anweisung verzweigt, sondern linear mit den Anweisungen für die nachfolgenden Fälle fortgefahren.

Abhilfe schafft die Anweisung **break**: Ihre Ausführung bewirkt, dass die Bearbeitung der **switch**-Anweisung sofort beendet und mit der ihr folgenden Anweisung fortgefahren wird.

Unser Beispiel müssen wir also so korrigieren:

```
switch (wochentag) {
    case SONNABEND:
        printf("Sonnabend\n");
        break;
    case SONNTAG:
        printf("Sonntag\n");
        break;
    ...
    case FREITAG:
        printf("Freitag\n");
        break;
    default:
        printf("Kennzahl unzuverlässig\n");
        break;
}
```

Ob man hinter der letzten Klausel eine **break**-Anweisung schreibt oder nicht, ist reine Geschmackssache, sie hat auf den Programmablauf keinen Einfluss.

Als weiteres, konkretes Beispiel sehen wir uns noch einmal die Berechnung von Fläche bzw. Umfang eines Kreises an, jetzt mit einer `switch`-Anweisung realisiert:

```
#include <stdio.h>

#define PI 3.14159...

int main(void) {
    int antwort;
    double r;

    printf("Bitte geben Sie den Radius ein: ");
    scanf("%lf", &r);
    while (getchar () != '\n')
        ;

    printf("Soll der Umfang (u/U) oder die Flaeche (f/F) "
           "des Kreises berechnet werden? ");
    antwort = getchar();
    switch (antwort) {
        case 'u':
        case 'U':
            printf("Umfang: %E\n", 2 * PI * r);
            break;
        case 'f':
        case 'F':
            printf("Flaeche: %E\n", PI * r * r);
            break;
        default:
            printf("Falsche Eingabe!\n");
    }

    return 0;
}
```

Quelltext 6.2: Berechnung von Kreisumfang oder Fläche mit `switch`

6.4 Die `while`-Schleife

Die `while`-Schleife haben wir praktisch bereits vollständig kennengelernt:

```
while (bedingung)
    anweisung
```

Solange der Ausdruck *bedingung* wahr – d.h. ungleich Null – ist, wird die folgende Anweisung ausgeführt. Anzumerken ist hier, wie bei der `if`-Anweisung: Formal besteht der Rumpf der Schleife aus genau einer Anweisung. Diese kann ein Block sein, muss es aber nicht.

6.5 Die `do`-Schleife

Die zweite Schleifenanweisung ist die `do`-Anweisung

```
do
    anweisung
while (bedingung);
```

die wir im Falle eines Blocks auch als

```
do {  
    anweisung  
} while (bedingung);
```

schreiben, um die Verwechslung der `while`-Klausel mit einer eigenständigen `while`-Anweisung zu verhindern.

Im Prinzip sind sich `do`- und `while`-Schleife sehr ähnlich. Der Unterschied zwischen den beiden Schleifen ist, dass bei einer `do`-Schleife sofort der Rumpf ausgeführt und erst danach die Bedingung überprüft wird, während bei einer `while`-Schleife die erste Operation überhaupt eine Überprüfung der Bedingung ist.

Trotzdem lassen sich beide Schleifenanweisungen ohne weiteres durcheinander ersetzen. So ist die Konstruktion

```
do  
    anweisung  
while (bedingung);
```

der Konstruktion

```
anweisung;  
while (bedingung)  
    anweisung
```

äquivalent, wobei die Formulierung als `do`-Schleife „natürlicher“ wirkt. Umgekehrt ist die Konstruktion

```
while (bedingung)  
    anweisung
```

der Konstruktion

```
if (bedingung)  
do  
    anweisung  
while (bedingung);
```

äquivalent, wobei hier die Formulierung als `while`-Schleife „natürlicher“ wirkt. Entsprechend sollte man im Einzelfall diejenige der beiden Schleifenanweisungen verwenden, die die „natürlichere“ Formulierung erlaubt.

In der Praxis kommen `while`-Schleifen häufiger als `do`-Schleifen vor, weil man in der Regel zunächst zu prüfen hat, ob der Schleifenrumpf überhaupt ausgeführt werden darf, bevor man ihn ausführt. Das Kopieren eines String ist insoweit eine Ausnahme, weil ein Zeichen, nämlich das abschließende Null-Zeichen, auf jeden Fall kopiert werden muss. Ein typisches Beispiel für `do`-Schleifen ist auch die Kommunikation mit dem Benutzer: Zunächst muss er seine Eingabe vornehmen – gibt er Unsinn ein, muss die Abfrage nach einer Ermahnung wiederholt werden.

Als Beispiel betrachten wir noch einmal die Aufgabe, vom Benutzer eine positive ganze Zahl zu erfragen. Als wir die Aufgabe zuerst gelöst hatten, hatten wir den „falschen“ Anfangswert Null für die Eingabevariable gesetzt, damit die Bedingung der Schleife anfangs den Wert *falsch* hatte. Darauf können wir jetzt verzichten. (Und auch sonst erlauben uns unsere erweiterten Kenntnisse eine ganze Reihe von Änderungen an der damaligen Lösung.)

```

#include <stdio.h>

int main(void) {
    int wert;                                /* Variablendekl.    */

    printf("Geben Sie die Zahl ein: ");

    do {
        if (scanf("%d", &wert) != 1) {      /* Zahl lesen        */
            printf("Keine Lust mehr?\n");    /* auf EOF/Fehler    */
            return 1;                        /* reagieren         */
        }
        if (wert <= 0)                      /* Eingabe zulaessig? */
            printf("Falsch - noch einmal! "); /* nein !           */
    } while (wert <= 0);

    printf("Einverstanden!\n");              /* ja !             */
    return 0;
}

```

Quelltext 6.3: Auswahl aus Alternativen (neu)

6.6 Die for-Schleife

Die dritte Schleifenanweisung in C ist die **for**-Anweisung

```

for (ausdruck1; bedingung; ausdruck2)
    anweisung

```

Der Ablauf lässt sich am einfachsten durch eine **while**-Schleife beschreiben:

```

ausdruck1;
while (bedingung) {
    anweisung
    ausdruck2;
}

```

Die *Werte* von *ausdruck1* und *ausdruck2* werden nicht verwendet. Man nutzt hier nur die Nebeneffekte, die bei der Berechnung der Ausdrücke auftreten, z.B. beim Auswerten des Zuweisungs- oder Inkrementierungsoperators. *ausdruck1* dient normalerweise der Initialisierung und *ausdruck2* der Aktualisierung der Schleifenvariablen, wie die Übersetzung zur **while**-Schleife bereits angedeutet hat.

In anderen Programmiersprachen gibt es in der Regel eine klare Unterscheidung, wann eine **while**- und wann eine **for**-Schleife einzusetzen ist:

- **for**-Schleifen können nur dann eingesetzt werden, wenn die Anzahl der Schleifendurchläufe bereits beim Eintritt in die Schleife bekannt ist, weil die Ausdrücke *ausdruck1*, *bedingung* und *ausdruck2* nur einmalig ausgewertet und aus ihnen, ebenfalls einmalig, die Anzahl der Schleifendurchläufe berechnet wird. Vor einer eventuellen Wiederholung wird dann nur noch diese (intern gespeicherte) Anzahl geprüft und modifiziert.
- Immer dann, wenn die Wiederholungsbedingung im Rumpf der Schleife modifiziert werden soll, *muss* man eine **while**-Schleife verwenden.

In C ist man zu dieser Unterscheidung zwar nicht gezwungen, wie wir gesehen haben. Es spricht aber manches dafür, sich freiwillig daran zu halten.

Typische Beispiele für den Einsatz der `for`-Anweisung sind in diesem Sinne Operationen auf Feldern. Das Beispiel, in dem das Skalarprodukt von zwei Vektoren berechnet wird, können wir etwa mit `for`-Anweisungen sehr viel kompakter formulieren, ohne dass die Lesbarkeit darunter leidet; ganz im Gegenteil – in diesem Fall verbessern die `for`-Anweisungen die Lesbarkeit sogar erheblich:

```
#include <stdio.h>

#define LAENGE 5

int main(void) {
    double v1[LAENGE], v2[LAENGE], prod;
    int i;

    printf("1. Vektor eingeben!\n");    /* 1. Vektor lesen */
    for (i = 0; i < LAENGE; i++)
        scanf("%lf", &v1[i]);

    printf("2. Vektor eingeben!\n");    /* 2. Vektor lesen */
    for (i = 0; i < LAENGE; i++)
        scanf("%lf", &v2[i]);

    prod = 0;
    for (i = 0; i < LAENGE; i++)        /* Skalarprodukt */
        prod += v1[i] * v2[i];          /* berechnen und */
                                        /* ausgeben */
    printf("Das Skalarprodukt ist: %f\n", prod);

    return 0;
}
```

Quelltext 6.4: Arbeit mit Vektoren: Skalarprodukt mit `for`-Schleife

Die Schrittweite einer `for`-Schleife muss nicht notwendig +1 (oder -1) sein. Durch

```
long potenz;

for (potenz = 1; potenz <= 100000; potenz *= 5)
    printf("%ld\n", potenz);
```

werden zum Beispiel alle Potenzen von 5 ausgegeben, die kleiner oder gleich 100000 sind.

Die drei Ausdrücke einer `for`-Klausel sind optional, können also auch weggelassen werden. Nur die Semikolons, die die Ausdrücke trennen, müssen stets geschrieben werden. Man könnte also zum Beispiel

```
long potenz = 1;

for (; potenz <= 100000; potenz *= 5)
    printf("%ld\n", potenz);
```

oder auch

```
long potenz = 1;
```

```
for (; potenz <= 100000;) {
    printf("%ld\n", potenz);
    potenz *= 5;
}
```

schreiben. Insbesondere im letzten Beispiel sollte man sich aber überlegen, ob nicht doch eine **while**-Schleife die angemessenere Realisierung ist.

Auch Endlosschleifen kann man erzeugen, die fehlende Bedingung wird hier als „wahr“ interpretiert:

```
for ( ; ; )
    anweisung
```

Gelegentlich hat man Schleifen mit zwei (oder mehr) Laufindizes. Wollen wir etwa die Reihenfolge der Komponenten eines Vektors umdrehen, so ist es zweckmäßiger, mit zwei Laufindizes zu arbeiten, als aus Länge und einem Index den anderen Index jeweils neu zu berechnen. Enthält die Variable *n* die Länge des Vektors, so können wir schreiben

```
for (i = 0, j = n - 1; i < j; i++, j--)
    h = v[i], v[i] = v[j], v[j] = h;
```

Hier zeigt sich, wie man den Kommaoperator für kompakte Formulierungen einsetzen kann.

Daneben bieten **for**-Anweisungen jede Menge Möglichkeiten zu Verstößen gegen den „guten Ton“! Zum Beispiel kann man die Laufvariable im Rumpf der Schleife ändern. Überlegen Sie sich einmal, was

```
for (i = 0; i < 100; i += 10) {
    printf("%d\n", i);
    i /= 2;
}
```

als Ausgabe liefert (oder probieren Sie es aus!).

6.7 Sprünge

Anweisungen zur Steuerung des Programmablaufs realisieren grundsätzlich *Sprünge* (*Verzweigungen*), d.h. Unterbrechungen des linearen Programmablaufs: Bei einer vollständigen **if**-Anweisung zum Beispiel wird eine der beiden Anweisungen „übersprungen“; bei einer Schleife wird ggf. vom Ende des Rumpfes zu seinem Anfang „zurückgesprungen“; die **return**-Anweisung bewirkt einen „Rücksprung“ in die rufende Routine.

Wenn man ein Programm liest, wird man bei **if**-, **switch**- oder Schleifenanweisungen an die Auswahl von Alternativen oder Schleifen denken und kaum an die in ihnen steckenden Sprünge. Solche Sprünge werden entsprechend auch als *implizite* Sprünge bezeichnet.

return- und **break**-Anweisung sind dagegen *explizite* Sprunganweisungen, d.h. ihr einziger oder zumindest wesentlicher Zweck ist das Ausführen eines Sprunges. Beide Anweisungen sind, wie auch die noch einzuführende **continue**-Anweisung, strukturbezogene Sprunganweisungen, d.h. das Ziel des Sprunges ergibt sich aus der Struktur des Programms und kann nicht frei gewählt werden.

Die vierte explizite Sprunganweisung, die **goto**-Anweisung, erlaubt dagegen die Wahl beliebiger Sprungziele innerhalb einer Funktion. Sie ist bei intensiver Verwendung ein sicheres Mittel, undurchschaubaren Code zu schreiben. Da man in C grundsätzlich ohne **goto**-Anweisungen auskommen kann, wird sie hier nicht weiter behandelt.

Auf die **return**-Anweisung kommen wir im Zusammenhang mit Funktionen noch einmal zurück.

6.8 Die **break**-Anweisung

Die **break**-Anweisung haben wir im Zusammenhang mit der **switch**-Anweisung kennengelernt. Außerdem darf sie in Schleifen aller Art verwendet werden und bewirkt dann die *sofortige* Beendigung der Schleife. Im Innern geschachtelter Schleifen wird immer nur die innerste Schleife beendet, während die umgebenden normal weiter abgearbeitet werden.

Nützlich ist die **break**-Anweisung in Schleifen vor allem dann, wenn das logische Ende in der Mitte des Schleifenrumpfes liegt. Ein Beispiel haben wir bereits kennengelernt, nämlich die Abfrage einer positiven ganzen Zahl. Hier muss in der Schleife zunächst gelesen und ggf. die Eingabe von EOF behandelt werden. War die Eingabe korrekt, ist die Schleife jetzt logisch zu Ende. Eingetragen werden muss aber noch die Behandlung unkorrekter Eingaben. Mit einer **break**-Anweisung kann man die Aufgabe so lösen:

```
#include <stdio.h>

int main(void) {
    int wert;                                /* Variablendekl. */

    printf("Geben Sie die Zahl ein: "); /* Zahl anfordern */

    do {
        if (scanf("%d", &wert) != 1) { /* Zahl lesen */
            printf("Keine Lust mehr?\n"); /* auf Abbruch */
            return 1;                    /* reagieren */
        }
        if (wert > 0)                    /* zulaessig ? */
            break;                       /* ja ! */
        printf("Auf Wiedersehen!");     /* nein ! */
    } while (1);

    printf("Einverstanden!\n");          /* alles klar ! */
    return 0;
}
```

Quelltext 6.5: Auswahl aus Alternativen (mit **break**)

Da das Ende der Schleife jetzt auch formal in der Mitte ihres Rumpfes liegt, spricht manches dafür, die Schleife als Endlosschleife zu formulieren.

Wenn das logische Ende einer Schleife mit ihrem formalen Ende übereinstimmt, kommt man in der Regel ohne größere „Klimmzüge“ auch ohne **break**-Anweisungen aus. Wollen wir etwa suchen, ob ein bestimmter Wert in einem Vektor enthalten ist, und ggf. bestimmen, an welcher Stelle er steht, so ist vielleicht die Formulierung

```
for (i = 0; i < N; i++)
    if (a[i] == x)
        break;
```

naheliegend. Lösen lässt sich die Aufgabe aber auch durch

```
for (i = 0; i < N && a[i] != x; i++)
    ;
```

wobei wir die Regeln für die Behandlung des Operators `&&` bewusst nutzen. Auch in vielen anderen Fällen kommt man ohne `break`-Anweisungen aus.

6.9 Die `continue`-Anweisung

Erlaubt es die `break`-Anweisung, eine Schleife sofort zu beenden, so erlaubt die `continue`-Anweisung, die momentane Abarbeitung eines einzelnen Schleifendurchlaufs zu unterbrechen und sofort zur Prüfung des Endkriteriums für die Schleife überzugehen; vom Ergebnis der Prüfung hängt es dann ab, ob die Schleife beendet oder ein weiterer Schleifendurchlauf ausgeführt wird.

Die `continue`-Anweisung wird nicht sehr oft verwendet. Sie lässt sich auch vermeiden. Sehen wir uns das an einer `while`-Schleife einmal an. Statt

```
while (bedingung1) {
    anweisungsfolge1
    if (bedingung2)
        continue;
    anweisungsfolge2
}
```

können wir ohne weiteres schreiben

```
while (bedingung1) {
    anweisungsfolge1
    if (!bedingung2) {
        anweisungsfolge2
    }
}
```

Man sieht hier auch direkt, was die `continue`-Anweisung leistet: Sie verringert die Schachtelungstiefe – mehr nicht.

6.10 Beispiel

Zum Abschluss des Abschnittes über die Ablaufsteuerung wollen wir uns ein etwas ausführlicheres Beispiel ansehen.

Es soll eine Dezimalziffer gelesen werden. Zu dieser Ziffer sollen alle positiven ganzen Zahlen kleiner als 100 ausgegeben werden, bei denen die Ziffer sowohl in der dezimalen Darstellung der Zahl selbst als auch in der ihres Quadrats vorkommt. Für die Ziffer 2 als Eingabe soll das Programm zum Beispiel diese Ausgabezeilen liefern:

```
23 529
25 625
27 729
32 1024
52 2704
82 6724
```

Es sollte klar sein, dass die obere Schranke als benannte Konstante deklariert wird. Die Wiederholung der Berechnung soll möglich sein, ohne dass dazu das Programm neu gestartet wird. Unzulässige Eingaben sollen (müssen!) zurückgewiesen werden.

Das Programm kann dann so aussehen:

```
#include <stdio.h>
#include <ctype.h>

#define GRENZE 100

int main(void) {
    int ziffer, c, i, j;

    printf("Geben Sie eine Ziffer ein, "
           "abgeschlossen mit ENTER: ");

    while (!isdigit(ziffer = c = getchar())
           || (c = getchar()) != '\n') {
        while (c != '\n')
            c = getchar();
        printf("Nur die Ziffern 0 bis 9 sind "
               "erlaubt! Nochmal: ");
    }
    ziffer -= '0';

    printf("Folgende Zahlen < %d erfuellen das "
           "Kriterium:\n", GRENZE);

    for (i = 1; i < GRENZE; i++) {
        for (j = i; j > 0 && j % 10 != ziffer; j /= 10)
            ;
        if (j == 0)
            continue;

        for (j = i*i; j > 0 && j % 10 != ziffer; j /= 10)
            ;
        if (j > 0)
            printf("%d^2 = %d\n", i, i * i);
    }

    return 0;
}
```

Quelltext 6.6: Ziffernübereinstimmungen

